

모델 보고서

🕒 생성일	2026년 9월 10일 오후 6:15
☰ 태그	

SOLAR 제출 모델 설명서

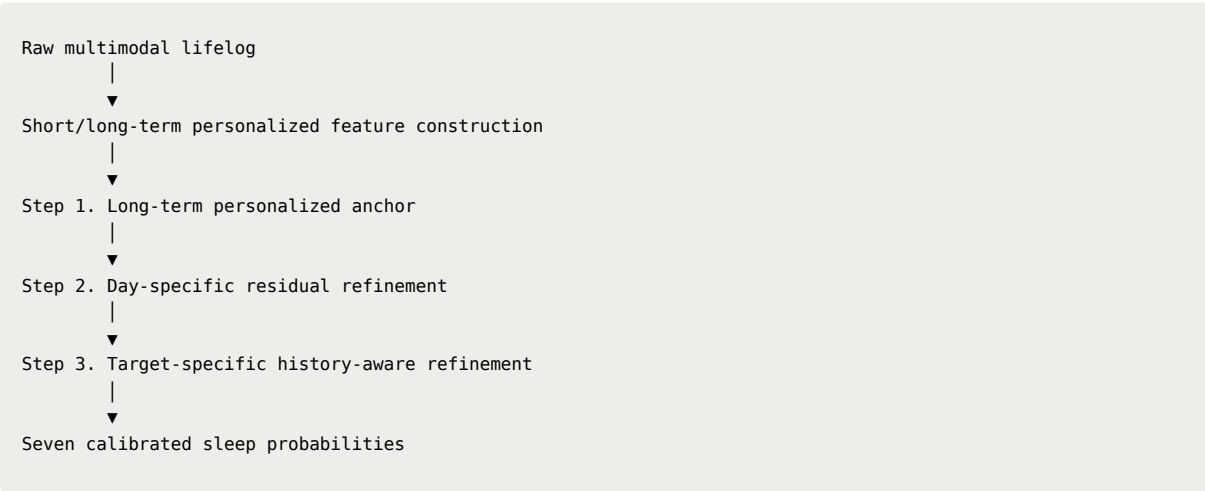
1. 모델 개요

SOLAR(Stepwise Sleep Prediction Framework via Long-Term Anchoring and Refinement)는 스마트폰과 웨어러블에서 수집한 멀티모달 라이프로그로 일별 수면 상태를 예측하는 개인화 프레임워크이다. 예측 대상은 주관적 수면 지표 Q1~Q3와 객관적 수면 지표 S1~S4이며, 각 목표가 1일 확률을 출력한다.

기존 라이프로그 기반 수면 예측은 당일 관측이나 짧은 기간의 통계에 주로 의존하고, 개인의 장기 상태와 당일 변화를 하나의 표현에서 동시에 학습하는 경우가 많다. 이 경우 안정적인 개인 생활 패턴과 특정 날짜의 일시적인 변화가 수면에 미치는 영향을 구분하기 어렵다.

SOLAR는 이 문제를 다음 세 단계로 분리한다.

1. 장기 개인 상태에 기반한 personalized temporal anchor 생성
2. 당일 센서 상태와 개인 기준 편차를 이용한 residual refinement
3. 같은 참가자의 시간적 연속성을 이용한 target-specific refinement



2. 예측 목표

목표	의미	값 1의 의미
Q1	기상 직후 주관적 수면 품질	개인 평균보다 양호
Q2	취침 전 신체 피로	개인 평균보다 피로가 낮음
Q3	취침 전 스트레스	개인 평균보다 스트레스가 낮음
S1	총 수면시간(TST)	권장 기준 충족
S2	수면 효율(SE)	권장 기준 충족
S3	입면 잠복기(SOL)	권장 기준 충족

목표	의미	값 1의 의미
S4	수면 중 각성시간(WASO)	권장 기준 충족

모든 목표는 이진 확률 예측으로 학습하며, 일곱 목표의 평균 Log-Loss를 평가 지표로 사용한다.

3. 데이터 및 특징 구성

3.1 멀티모달 입력

다음과 같은 스마트폰·웨어러블 센서 스트림을 사용한다.

- 스마트폰 활동, 화면 상태, 충전 상태 및 조도
- GPS 위치와 이동 속도
- Wi-Fi 및 Bluetooth 주변 기기
- 앱별 사용 시간
- 웨어러블 심박수와 조도
- 걸음 수, 보행 거리, 속도 및 소모 칼로리

비동기 센서 이벤트는 참가자와 시각을 기준으로 정렬하고, 수면 전후의 생활 맥락을 표현할 수 있도록 저녁, 취침 전, 심야, 수면 추정 구간, 기상 전후 및 아침 구간으로 나누어 집계한다.

각 구간에서 평균, 표준편차, 분위수, 사분위 범위, MAD, 변화량, 추세, 자기상관, 이벤트 간격, 수집 밀도와 결측률 등을 계산한다. 화면이 장시간 꺼진 구간은 잠재적인 수면 구간으로 사용하고, 해당 구간 내부의 화면 활성화는 수면 분절의 대리 특징으로 활용한다.

3.2 단기 및 장기 개인화 특징

- 단기 특징: 최근 1~7일의 센서 상태와 변화
- 장기 특징: 최근 14~28일 통계, 누적 평균·분산, 동일 요일 평균
- 대조 특징: 단기 평균과 장기 평균의 차이
- 개인화 특징: 현재 상태와 해당 참가자의 평상시 상태 간 편차
- 이력 특징: 최근 상태, 연속 길이, 상태 전환 횟수
- 신뢰도 특징: 센서별 결측, 수집 범위 및 과거 수집 안정성

학습 행의 이력 특징에는 현재값이나 미래값이 포함되지 않도록 참가자별 시간순 정렬 후 `shift(1)` 을 적용한다.

4. 모델 구조

4.1 Step 1: Long-term Personalized Temporal Anchor

Step 1은 참가자의 장기 생활 상태와 반복되는 시간 패턴을 이용해 목표별 기준 확률을 생성한다. 이를 위해 개인 이력, 상태전이, calendar prior, 날짜 거리, 장기 센서 상태 및 tabular model 등 서로 다른 성격의 prediction source를 OOF 방식으로 생성한다.

참가자 i , 날짜 t , 목표 k 에 대한 anchor는 다음과 같다.

$$\hat{p}_{i,t,k}^{(1)} = F_k \left(s_{i,t,k}^{(1)}, \dots, s_{i,t,k}^{(M_k)} \right)$$

여기서 $s_{i,t,k}^{(m)}$ 은 개별 prediction source이며, $F_k(\cdot)$ 는 목표별 stacking 함수이다. 유효한 source가 목표마다 다를 수 있으므로 source 선택과 stacking을 Q1~S4별로 독립적으로 수행한다.

구현에서는 XGBoost, Logistic Regression, Ridge Regression, greedy source blend 및 이들의 안정화 양 상블을 비교한다. 전체 OOF와 참가자별 마지막 시간 구간의 Log-Loss를 함께 고려하여 목표별 anchor를 결정한다.

4.2 Step 2: Day-Specific Personalized Residual Refinement

Step 1의 anchor는 개인의 안정적인 기준 상태를 표현하지만, 특정 날짜의 활동, 야간 스마트폰 사용, 심박수, 이동량 및 환경 변화는 충분히 반영하지 못할 수 있다. Step 2는 목표를 처음부터 다시 예측하지 않고 Step 1이 설명하지 못한 residual을 학습한다.

$$r_{i,t,k} = y_{i,t,k} - \hat{p}_{i,t,k}^{(1)}$$

$$\hat{p}_{i,t,k}^{(2)} = \text{clip} \left(\hat{p}_{i,t,k}^{(1)} + \eta_k \hat{r}_{i,t,k}, \epsilon, 1 - \epsilon \right)$$

Residual 입력에는 당일 시간대별 센서 상태, 개인 장기 평균과의 편차, 최근 변화, 센서 결측, 화면 기반 수면 구간과 수면 분절 대리 특징이 포함된다.

Ridge, Huber 및 ExtraTrees residual model을 비교하며, 작은 η_k 후보를 사용해 anchor가 과도하게 변경되지 않도록 한다.

최종 제출 구성은 서로 다른 특징 수를 사용하는 세 residual run을 사용한다.

- S01_K128: 목표별 상위 128개 residual 특징
- S00_BASE_V7: 목표별 상위 32개 residual 특징
- S01_K32: 목표별 상위 32개 residual 특징

세 예측은 목표별로 logit 공간에서 가중 결합하여 E02 anchor를 생성한다.

4.3 Step 3: Target-Specific History-Aware Refinement

목표별 시간 의존성이 서로 다르므로, 검증에서 효과가 확인된 목표에만 같은 참가자의 temporal-neighbor prior를 적용한다. 가까운 날짜에 더 큰 가중치를 부여하되 단기·장기 시간 척도를 함께 사용한다.

최종 제출에서는 Q2, Q3, S2에 temporal-neighbor refinement를 적용하고, S4에는 확률 calibration을 적용한다. 이후 OOF 검증으로 고정된 Q1 source gate, Q2 source correction, S4 residual gate 및 Q2-Q3 공동 상태에 따른 제한적 Q3 보정을 순서대로 수행한다.

이 마지막 target-wise calibration은 SOLAR의 핵심 방법론인 anchor-residual-history 구조를 유지하면서 목표별 확률 분포 차이를 보정하는 제출 모델의 구현 요소이다.

5. 학습 및 검증 방법

참가자별 기록을 날짜순으로 정렬하고 이전 시간 구간으로 이후 구간을 예측한다. 참가자별 마지막 약 25% 기록은 별도의 last-block 검증 구간으로 사용한다.

후보 모델은 전체 OOF Log-Loss와 last-block Log-Loss를 함께 평가한다. 전체 성능이 개선되더라도 최근 구간이나 일부 참가자에서 손실이 크게 증가하는 후보는 선택하지 않거나 보정 강도를 낮춘다.

확률 안정화를 위해 다음 방법을 사용한다.

- OOF 기반 stacking과 residual 학습
- 확률 clipping 및 shrinkage

- 목표별 residual strength 제한
- logit-space blending
- temperature calibration
- source 간 불일치와 센서 수집률에 따른 gate

6. 연구의 참신성

장기 상태와 당일 변화의 명시적 분리

장기 이력과 당일 센서를 하나의 모델에서 직접 학습하지 않고, 장기 상태로 anchor를 만든 뒤 당일 정보가 residual만 수정하도록 역할을 분리한다. 이를 통해 안정적인 개인 경향과 일시적인 생활 변화를 구조적으로 구분한다.

개인 기준 상대 변화 모델링

동일한 센서값도 참가자마다 의미가 다를 수 있다는 점을 고려해 절대값뿐 아니라 현재 상태와 개인 누적 평균, 동일 요일 평균 및 장기 센서 상태의 차이를 학습한다.

목표별 선택적 결합

주관적 Q 계열과 객관적 S 계열의 생성 과정이 다르다는 점을 반영하여 prediction source, stacking recipe, residual feature, 보정 강도 및 temporal refinement를 목표별로 선택한다.

시간 누수를 고려한 개인별 검증

무작위 분할 대신 참가자별 시간순 분할과 last-block 평가를 사용하고, 이력 특징에는 현재 행이 포함되지 않도록 구성한다.

7. 코드 구성

- `run.py`: 결정론적 환경 설정과 실행 진입점
- `solar_pipeline.py`: 논문의 세 단계에 대응하는 명시적 orchestration
- `solar_core.py`: 원시 센서 처리, source model, stacking 및 보정 구현
- `environment.yml`: Python 3.12 기반 Conda 환경 정의
- `requirements.txt`: 재현을 위해 버전을 고정한 Python 의존성
- `setup_conda.sh`: Conda 환경 생성과 Python 의존성 설치 자동화
- `run_solar.sh`: 외부 Python 환경을 차단하고 프로젝트 Conda 환경에서 실행

7.1 사전 요구사항

실행 머신에는 Anaconda 또는 Miniconda와 `unzip` 이 설치되어 있어야 한다.

다음 명령으로 Conda 사용 가능 여부를 확인한다.

```
conda --version
```

7.2 압축 해제

제출 ZIP을 내려받은 폴더에서 다음과 같이 압축을 해제하고 프로젝트 폴더로 이동한다.

```
unzip submit_final_end2end.zip
cd submit_final_end2end
```

7.3 Conda 환경 및 의존성 설치

다음 스크립트는 프로젝트 내부에 `.conda-env` 환경을 만들고 `requirements.txt`에 고정된 패키지를 모두 설치한다. 이 단계에서는 모델 학습을 시작하지 않는다.

```
chmod +x setup_conda.sh
chmod +x run_solar.sh
./setup_conda.sh
```

설치가 끝나면 환경을 활성화한다.

```
conda activate "$PWD/.conda-env"
```

현재 셸에서 `conda activate` 사용이 설정되어 있지 않다는 오류가 발생하면 다음을 한 번 실행한 뒤 다시 활성화한다.

```
source "${conda info --base}/etc/profile.d/conda.sh"
conda activate "$PWD/.conda-env"
```

7.4 전체 코드

원시 센서 데이터부터 전체 모델을 다시 학습하는 권장 명령은 다음과 같다. 이 스크립트는 Conda 활성화 여부와 관계없이 프로젝트 내부 `.conda-env`를 직접 사용하며 외부 `PYTHONPATH`, `PYTHONHOME` 및 사용자 패키지를 차단한다.

```
./run_solar.sh
```

환경을 직접 활성화한 경우에는 다음 명령도 동일하다.

```
python run.py --no-frozen
```

중단된 동일 환경에서 특징 캐시를 재사용하여 이어서 실행하려면 다음 명령을 사용한다.

```
./run_solar.sh --resume --reuse-feature-cache
```

기본 실행에서 제외된 MIS-LSTM 후보까지 포함하려면 다음과 같이 실행한다.

```
./run_solar.sh --include-mis-lstm
```

7.5 실행 환경

사용한 실행 환경은 Conda와 POSIX 셸을 사용할 수 있는 **64비트 Linux** 환경이다.

GPU가 없는 머신에서는 PyTorch가 CUDA를 사용하지 않으며 코드가 지원하는 CPU 경로로 실행될 수 있지만 학습 시간이 크게 증가할 수 있다.

7.6 결과 확인

학습이 정상적으로 종료되면 최종 제출 파일은 다음 위치에 생성된다.

```
data/submissions/submission_final.csv
```